

Programming Lab 1 (CS21201)

M.Tech. I Semester (2026-27)

Assignment:- 02

Question No 1:

PhD Genealogy Tree

Your assignment is to implement an “academic genealogy” data type to model professors and their advising relationships. A PhD degree is obtained under the mentorship of a professor, so these mentoring relationships can be viewed as a tree in which each student is a child node of their mentor’s node.

In order to meet the requirements of a tree, only a student’s relationship with their primary advisor will be tracked (that way, each child node only has a single parent). A PhDTree represents a general tree of professors-each node may have any number of children. The tree is represented recursively-a “node” can be considered as a subtree and is therefore represented as its own PhDTree.

Each node includes the name of a former PhD student and the year in which they earned their degree. These details, along with their advisor’s name, will be provided in a CSV file, which your program will read in order to construct the tree in memory. The professor at the root of the tree will not have an advisor listed; their academic ancestors will not be considered by your program.

Given this tree representation, you will be able to compute some interesting properties of the genealogy, such as finding the most recent ancestral advisor shared by two professors. Try to implement a program that will provide an interactive interface for querying such properties.

Basic Operations to Create PhDTree

Queries:

contains(): Determine whether the professor specified in the argument is contained in the PhDTree. The result should indicate whether the professor is contained or not contained.

size(): Count the number of professors contained in a subtree rooted at the professor specified in the argument, or in the whole tree if no argument is provided.

findAdvisor(): Searches for a target professor but returns the professor in the parent node instead.

numLeaves(): A counting operation for a subtree (originating with the argument node) that accumulates a result from all of its children’s results.

findProlificMentor(): Counts the advisees of each advisor and identifies the advisor with the highest count.

findAcademicLineage(): Lists professors from the earliest ancestor to the current node.

commonAncestor(): Finds the most recent common academic ancestor of two professors. Even if two professors can both trace their advisors back to Isaac Newton, at some point their lineages

are likely to diverge. The purpose of the `commonAncestor()` method is to find this divergence point.

Using a preorder traversal with a `SortedSet` of children, the lines of the output should come in an order such that advisors precede their advisees.

Reflection on the Assignment

Time. How much time (in hours) was spent on the assignment? (Please only write a number here.)

Implementation strategy. Describe how you went about implementing the assignment and how the work was divided between the partners.

Testing strategy. How did you perform testing and design test cases?

Known problems. Are there any things that do not work?

Comments. Do you have any comments on the assignment?

Be thinking about these points as you work. When you finish, write your responses.

Question No 2:

AVL Tree

An AVL tree is a binary search tree which has the following properties:

i. An AVL tree is a balanced binary search tree. In an AVL tree, the balance factor of every node is either -1, 0, or +1.

$$\text{Balance Factor} = \text{Height of Left Subtree} - \text{Height of Right Subtree}$$

ii. Every subtree is an AVL tree.

Write a menu-driven program in C/C++ to implement an AVL Tree. The program must show basic functionalities such as insertion, deletion, and display of the tree, while following the basic properties of an AVL tree.